
Term Project

Project Specification

Instructor	Sang Kil Cha
Proposal Due	May 4, 2026
Final Due	June 1, 2026
Peer Review Due	June 8, 2026
Submission	https://softsec.kaist.ac.kr:20200/

1 Overview

In this term project, you will design and implement a simple game. It does not need to be complex: a console-based game is perfectly acceptable, and a small game with clear, well-defined behavior is better than an ambitious one with unclear or unfinished behavior. Your work will not be graded on the complexity of the game. See Section 8 for an example of a good term project based on a simple console-based tic-tac-toe game.

Before implementing the game, write your proposal. This proposal is a requirements document. It should specify what your game will do, what the user can do, what the system will do in response, and how the game ends. This is a requirements engineering task: you are expected to think carefully about observable behavior before writing the implementation.

After your proposal is submitted, you will implement the game described in that proposal. Your final score will be based mostly on how faithfully your final implementation satisfies the requirements that you proposed.

This project therefore has *two distinct deliverables*.

- **Proposal** (due May 4, 2026): two submitted items, a public GitHub repository and a PDF requirements document that defines the game you plan to build.
- **Final** (due June 1, 2026): the completed F# game in the public GitHub repository, together with a `README.md` file with instructions for how to run the game.

2 Project Theme and Scope

Your project should be a game. It may be text-based, graphical, web-based, or implemented in another form. The implementation must be written in F# and use .NET 10.

A simple game with precise requirements and a faithful implementation, meaning one that matches the proposed requirements, is a good project. Complexity is not graded.

If your project uses supporting files, such as data files, images, or build configuration files, include them in the repository. The submitted project should be runnable from the public GitHub repository using the instructions in your `README.md` file.

3 Proposal: Requirements Document

Your proposal must include a requirements document submitted as a PDF. This document defines the project you plan to implement. It should be specific enough that another person can evaluate whether your final game satisfies it. See Section 8 for an example of how such a requirements document can be written.

Good requirements are concrete, testable, and observable. For example, in a Tic-Tac-Toe game, “the game is fun” is not a good requirement because it is subjective, but “the user selects an empty square by entering a number from 1 to 9” is a good requirement because a reviewer can check whether the implementation behaves that way.

4 Submission Instructions

You will submit your project to <https://softsec.kaist.ac.kr:20200/>.

4.1 Proposal Submission

For the proposal, you must submit two things by May 4, 2026:

1. a public link to your GitHub repository, and
2. a PDF requirements document.

★ **The repository may be empty at the proposal stage**, but it must already exist by the proposal deadline and be publicly accessible to TAs and other students without requesting permission.

4.2 Final Submission

By the final deadline (June 1, 2026), you do not need to make a separate new submission to HotCRP. Instead, continue working in the same public GitHub repository to complete the game described in your proposal.

At the final deadline, the repository should contain your completed F# game and a `README.md` file that explains how to run the game.

It is your responsibility to make sure that the game is runnable from the requirements and instructions described in `README.md` alone, since this is part of the peer review grading. We strongly suggest testing your project on a clean machine installation and avoiding complex dependencies whenever possible.

If you wish to change your requirements document after your proposal submission, record the changes in your `README.md` file and explain why those changes were made. This change log should note what changed and the justification for each change. For example, removing a planned sound effect may be justifiable if you explain that a required dependency did not work reliably or that embedding sound in the final project turned out to be infeasible. In contrast, changing a major requirement without such a concrete reason is not a justifiable change.

4.3 HotCRP Submission Steps

1. Sign in at <https://softsec.kaist.ac.kr:20200/> using your KAIST email address.
2. You will receive an email within 24 hours from the release of the project containing your account information, including your password. Change the password as soon as possible. Contact the TAs if you do not receive the email within 24 hours.
3. Create your submission in HotCRP.
4. Your account will have the following information: Name, Email, and Affiliation. Your affiliation should be your student ID by default. **Do not change this information in the account settings or in your submission. Keep submission authors as default, and do not add any other authors.**
5. Provide both the URL of your public GitHub repository (may be empty at the proposal stage) and your PDF requirements document in the same submission.
6. Submit only one submission per student.

5 Grading

This project will be peer reviewed. Reviewers, meaning other students, will grade your project mainly on the submitted requirements, whether later requirement changes are justified, and whether the final implementation satisfies those requirements.

Below is the grading rubric. Reviewers will assign scores according to the following scales.

Category	Points	Criteria
Runnable from README	1	The game is not runnable from the README.md instructions and stated requirements alone.
	2	The game is runnable from the README.md instructions and stated requirements alone.
Requirement changes and justification	1	There are non-justifiable changes in the requirements after submission. Example: a major feature is removed or changed without a concrete reason.
	2	There are no changes, or all requirement changes are justifiable. Example: a planned sound effect is removed from the requirements because the necessary dependency was unreliable or embedding sound turned out to be infeasible.
Requirements testability	1	Requirements are not testable for evaluation. Example: "the player wins after collecting 10 coins" is testable, but "the game is exciting" is not.
	2	Requirements are partially testable for evaluation.
	3	Requirements are fully testable for evaluation.
Implementation faithfulness to requirements	1	Implementation is not faithful to requirements, or the requirements are not testable.
	2	Implementation is partially faithful to requirements.
	3	Implementation is fully faithful to requirements.
Conformance to instructions	1	The submission does not follow one or more of the required project instructions listed below.
	2	The submission follows all the project instructions.

5.1 Peer Evaluation

A single-blind peer review process will be used through HotCRP. Peer evaluation will be opened after the final deadline.

A *single-blind* review process means that other student reviewers can see the authors of the submission, but authors do not see who reviewed their work. You should write reviews professionally and fairly.

Each submission will receive four peer reviews, and each student will randomly be assigned four other students' projects in HotCRP to review. For each assigned project, you will read the submitted requirements PDF and find the repository link, visit the public GitHub repository, read the requirements PDF and README.md, run and inspect the game, and fill in the grading criteria shown in the grading section above.

When scoring "Conformance to instructions," the relevant instructions are:

- the implementation should be written in F# and use .NET 10,
- the repository should include a README.md, and
- if an LLM is used, the project should include the required attribution in the README.md as described in Section 7.

For the LLM attribution item, reviewers do not need to be extremely thorough; we only expect them to consider obvious cases.

Although we expect no further repository changes after the final deadline, we do not enforce this strictly. Instead, reviewers should assign scores based on the state of the repository at the time of review. Any changes made after the deadline or after the review is completed will not be considered for scoring.

You are also required to nominate at least one of the games you review as a candidate for the Best Game Award. This nomination should be used only for a game that receives a perfect score according to the grading criteria and that you enjoyed playing the most. Reviewer nominations will be considered by the instructors as part of the Best Game Award selection process; see Section 6.

Please note that if you do not submit all four assigned reviews by the review deadline, points will be deducted from your project score.

5.2 Using HotCRP for Peer Review

To use HotCRP for peer review:

1. Open the list of assigned reviews in HotCRP.
2. For each assigned submission, open the submitted requirements PDF and repository link, inspect the project, and then open the review form.
3. Fill in the grading criteria shown in the grading section above for each of the four assigned projects.
4. Nominate at least one reviewed game as a Best Game Award candidate, but only if it received a perfect score from you and you enjoyed playing it.

5. Submit each completed review in HotCRP by the review deadline.

During the review period, and before the final grading is revealed to the authors, reviewers will be able to see other reviews for the same submission and discuss them with each other.

If there are scores from other reviewers that you do not agree with, we encourage you to discuss them with the other reviewers and try to reach a mutually agreed score. For example, you may think that the `README.md` explanation is sufficient to run the program while another reviewer may think that it is not. This is an important step in the peer review process to ensure the fairness and consistency of the reviews.

6 Best Game Award

The instructors will select at least one Best Game Award winner from all submissions. As explained in class, a Best Game Award winner will receive an A+ for this course regardless of any other scores.

Reviewer nominations will be considered as part of the judging process, but the instructors will not be limited to those nominations when making the final decision. The winning game or games will be posted in the hall of fame (GitHub).

7 Use of Large Language Models

You may use a large language model (LLM) for this project, but **you are not required to use one**. If you do, you are still responsible for the final requirements document and implementation. You should understand the code you submit, and the final program must satisfy your proposal.

If you use an LLM in any way while developing this project, you must add a section to your project's README describing your experience working with the LLM. This section should explain:

- what you used the LLM for,
- what you had to manually change or reprompt because the LLM did not understand your first prompt, and
- the main point that the LLM was not able to do correctly.

8 Example: CLI Tic-Tac-Toe

This section shows an example of an appropriate requirements document. The game is intentionally simple. This is a good project because its behavior is clear and easy to evaluate.

8.1 Example Requirements Document

Project Title: CLI Tic-Tac-Toe

Overview: This project is a command-line tic-tac-toe game where the user plays against a random enemy. The user plays as O, and the enemy plays as X.

Requirements:

1. The user will see a 3x3 tic-tac-toe board in the terminal.
2. The user will make a move by selecting an empty square from 1 to 9.
3. If the user selects an empty square, O will be placed in that square. Otherwise, the user will be asked to retry the input.
4. After the user makes a move, it becomes the enemy's turn. The enemy will always choose a random empty square and place X in that square.
5. The user and the enemy will take turns making moves.
6. When the user gets three Os in a row in any direction, the user wins. When the enemy gets three Xs in a row in any direction, the enemy wins. If the board is full without the user or the enemy winning, the game is tied.

Example Interaction: The game prints a board with numbered empty squares. The user enters 5. The game places O in the center square. The enemy randomly chooses an empty square and places X there. The game prints the updated board and asks the user for the next move.

8.2 Example LLM Usage

For illustration, we provide an example of a game fully written by an LLM system. The `requirements.md` file used is identical to the requirements document in Section 8 above. The prompt given to the LLM was:

Write a command line tic-tac-toe game using F# (dotnet 10). You should satisfy `requirements.md`. After you implement it, you should test its behavior conforms requirements.

The resulting implementation can be found at github.com/KAIST-CS20200/project-example. Refer to that repository for further details.

9 FAQ

Can I use an LLM? Yes. You may use an LLM, but you are not required to. If you do, you are responsible for understanding and verifying the result, and you must acknowledge the LLM assistance in your `README.md`. See Section 7 for the required README discussion of your experience using the LLM.

What if I do not have access to a strong LLM? You are not required to use an LLM, let alone a strong LLM. Note that as a student, you may also be eligible for a free Copilot subscription through GitHub Education, but again, you are not required to create a complex game or use an LLM.

How complex should my project be? It does not need to be complex. Complexity is not graded.

Am I allowed to use external libraries? Yes. You are allowed to use external libraries.

What is the proposal? The proposal is your requirements document submitted as a PDF. It describes the game you plan to implement and will be used later to evaluate your final project.

What do I submit to HotCRP? For the proposal, submit a public GitHub repository (may be empty) and a PDF requirements document. The repository must be accessible to everyone. By

the final deadline, you do not have to submit anything new. However, the same repository must now be complete with your F# implementation and a README.md file explaining how to run the game. By the peer review deadline, you must have completed your peer reviews in HotCRP.

What if I change my idea after submitting the proposal? You may change your requirements after submitting the proposal, but you must document those changes in your README.md file and explain why they were necessary or reasonable. For example, removing a minor visual feature because the final game is console-based can be justifiable. In contrast, changing the project into a substantially different game without strong justification is not.

10 Questions

Ask all clarification questions through GitHub Discussions. Before posting a new question, search the existing discussions to avoid asking a duplicate question; otherwise, as per class policy, asking duplicate questions will deduct your score.